

MATH 231 – Linear Algebra I

The Computational Unit, Part 1 ■ Counting Operations

Kiely Hall 326 ■ Instructor: Kevin Bedoya

About this document. This opens a new unit, on the computational side of the subject: how much work a calculation actually costs, how numbers are really stored, and how the algorithms of this course are built and analysed.

A common confusion (Section numbers restart). This unit numbers its sections from 1 again — they do *not* continue from the vector unit. So a reference to “§1.3” below means §1.3 *of this unit*. Whenever we point back at the earlier material we will say so explicitly, as in “the vector unit, §11.3”. Both units have a §11.3, and they are different sections.

Assumed from the vector unit. Vectors in $\mathbb{R}^n$ and componentwise addition (§7 there); the dot product $\langle \mathbf{u}, \mathbf{v} \rangle = \sum_i u_i v_i$ (§11.3); the Euclidean norm (§2); the projection $\text{proj}_{\mathbf{u}}(\mathbf{v})$ (§12.4); the identity $\langle \mathbf{u}, \mathbf{u} \rangle = \|\mathbf{u}\|^2$ (§11.6); orthogonality and cosine similarity (§11.4, §11.7); and linear dependence (§18.4).

What you should be able to do afterwards. Say what an *operation* is; count exactly how many operations vector addition, the dot product, the norm and the projection require in $\mathbb{R}^n$; recognise redundant work in a formula and remove it; and explain why an exact count is worth having.

1. Counting operations

1.1. A first count: adding two vectors

Let $\mathbf{u}, \mathbf{v} \in \mathbb{R}^n$ and set $\mathbf{w} = \mathbf{u} + \mathbf{v}$. Here is a question we have never asked before:

How many calculations must a machine perform in order to produce $\mathbf{w}$?

Before answering, one condition. Addition of vectors — like every vector operation in this course — is defined only when both inputs have the *same* dimension. There is no sum of a vector in $\mathbb{R}^3$ and a vector in $\mathbb{R}^5$: the definition matches up first coordinate with first, second with second, and would run out. So we may take both to have exactly n coordinates.

Now count. Addition is componentwise, so

$$\mathbf{w} = [u_1 + v_1, u_2 + v_2, \dots, u_n + v_n].$$

Each slot of the answer requires exactly one addition of two real numbers, and there are n slots. Nothing else happens. So producing $\mathbf{w}$ costs

n operations

and the same count holds for $\mathbf{u} - \mathbf{v}$, which differs only in replacing each addition by a subtraction.

1.2. What counts as an operation

Definition 1.1 (Operation). In a computational setting, an **operation** is a single calculation carried out on scalar quantities by one operator — one addition, one subtraction, one multiplication, one division, one square root.

Two pieces of vocabulary we will use constantly: a scalar addition is an **add**, and a scalar multiplication is a **mult**. So the count above reads: vector addition in $\mathbb{R}^n$ costs n adds.

Remark. Counting every operation as costing “one” is a simplification, and a deliberate one. On real hardware a multiplication is somewhat slower than an addition, a division is several times slower again, and a square root slower still. We ignore that for now because the first thing worth knowing about a computation is *how the count grows with n* , and that pattern is the same whichever weights you attach. We will revisit the assumption when it starts to matter.

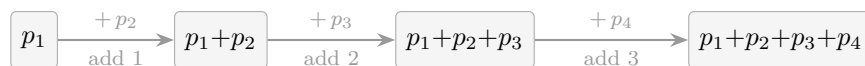
1.3. The dot product

Now take $\langle \mathbf{u}, \mathbf{v} \rangle$. Write out what the definition asks for:

$$\langle \mathbf{u}, \mathbf{v} \rangle = u_1 v_1 + u_2 v_2 + \cdots + u_n v_n.$$

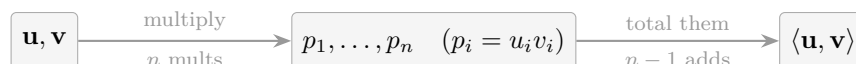
Two distinct stages are visible. First, n products $u_i v_i$ must be formed — that is n mults. Then those n numbers must be added into a single total.

How many adds does it take to total n numbers? Not n . Adding is a *binary* operation, so each add consumes two numbers and returns one, reducing the count of outstanding numbers by exactly one. Starting with n numbers and finishing with 1 therefore takes $n - 1$ adds.



4 numbers, 3 adds — in general n numbers, $n - 1$ adds

So the dot product runs in two stages:



Adding the two stages,

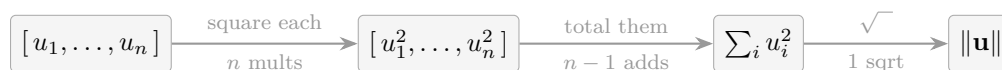
$$\langle \mathbf{u}, \mathbf{v} \rangle \text{ costs } n + (n - 1) = 2n - 1 \text{ operations}$$

1.4. The norm

Next, $\|\mathbf{u}\|$. Written out term by term,

$$\|\mathbf{u}\| = (u_1^2 + u_2^2 + \cdots + u_n^2)^{1/2}.$$

Now three stages. Each u_i^2 is the product $u_i \cdot u_i$, so squaring all n coordinates is n mults. Totalling those n squares is $n - 1$ adds, by the counting argument of §1.3. And finally one square root is taken, of a single scalar — one operation, no matter how large n is, because by that point only one number is left.



Adding the three stages,

$$\|\mathbf{u}\| \text{ costs } n + (n - 1) + 1 = 2n \text{ operations}$$

Remark. It is worth noticing where the 1 sits. The square root is applied *once*, at the very end, to a scalar — not once per coordinate. Miscounting this is the most common error in a first operation count: the stages of a computation operate on different numbers of quantities, and you must track how many are still in play at each stage.

1.5. The projection

Now something with several moving parts. Recall from the vector unit, §12.4,

$$\text{proj}_{\mathbf{u}}(\mathbf{v}) = \frac{\langle \mathbf{u}, \mathbf{v} \rangle}{\|\mathbf{u}\|} \cdot \frac{\mathbf{u}}{\|\mathbf{u}\|}.$$

The hard work is already done: we know what a dot product and a norm cost. What remains is to take the formula apart and add up the pieces, being careful at each step about whether the quantity involved is a scalar or a vector.

1. $\langle \mathbf{u}, \mathbf{v} \rangle$ — a dot product, so $2n - 1$ operations.
2. $\|\mathbf{u}\|$ — a norm, so $2n$ operations. The formula asks for it *twice*, once in each factor, so as written that is $4n$.
3. $\langle \mathbf{u}, \mathbf{v} \rangle / \|\mathbf{u}\|$ — careful here. Both of these are *scalars*, so this is a single scalar division: 1 operation.
4. $\mathbf{u} / \|\mathbf{u}\|$ — here $\mathbf{u}$ is a *vector*, and dividing it by a scalar is componentwise. That is n divisions, so n operations.
5. Finally the scalar from step 3 multiplies the vector from step 4, again componentwise: n mults, so n operations.

Adding them up,

$$(2n - 1) + 4n + 1 + n + n = 8n,$$

so the formula *as written* costs $8n$ operations.

1.6. Doing it better

Is there anything wrong with that count? Mathematically, no — $8n$ is the honest cost of evaluating the expression exactly as it appears. But to anyone who has written code, the problem is glaring.

First saving: stop computing the same thing twice

Why did we compute $\|\mathbf{u}\|$ *twice*? The two occurrences are the same number. Compute it once, keep it, and use it in both places. That deletes one of the two $2n$ charges:

$$(2n - 1) + 2n + 1 + n + n = 6n.$$

Nothing about the mathematics changed — only the bookkeeping. This is the single most common optimisation there is, and it has a name in programming: *common subexpression elimination*.

Second saving: do not take a square root you are going to square

Look at what $\|\mathbf{u}\|$ is used for. It appears twice, and both times it divides — so what the formula really uses is $\|\mathbf{u}\|^2$. And by the vector unit's §11.6,

$$\|\mathbf{u}\|^2 = \langle \mathbf{u}, \mathbf{u} \rangle,$$

which is a dot product: $2n - 1$ operations, with *no square root at all*. Rewriting the projection in the equivalent form already given in §12.4,

$$\text{proj}_{\mathbf{u}}(\mathbf{v}) = \frac{\langle \mathbf{u}, \mathbf{v} \rangle}{\|\mathbf{u}\|^2} \mathbf{u},$$

the count becomes

$$\underbrace{(2n-1)}_{\langle \mathbf{u}, \mathbf{v} \rangle} + \underbrace{(2n-1)}_{\langle \mathbf{u}, \mathbf{u} \rangle} + \underbrace{1}_{\text{one scalar division}} + \underbrace{n}_{\text{scale } \mathbf{u}} = 5n - 1.$$

Notice the third saving hiding in that line. The old version made two separate passes over $\mathbf{u}$ — dividing by the norm, then multiplying by the scalar — costing $n + n$. Combining the two scalars into one *before* touching the vector reduces that to a single pass of n mults.

Version	Cost	What changed
As written	$8n$	—
Reuse $\ \mathbf{u}\ $	$6n$	compute the norm once, not twice
Use $\ \mathbf{u}\ ^2 = \langle \mathbf{u}, \mathbf{u} \rangle$	$5n - 1$	no square root; one pass over $\mathbf{u}$

For $n = 1000$ that is 8000 operations against 4999 — the same answer, at a little over half the cost. And the square root, which we noted in §1.2 is among the slowest operations available, has been removed entirely.

1.7. Why count at all

A fair question at this point is why any of this matters. The formulas are correct either way, and a modern machine will evaluate all three versions faster than you can notice.

The reason is that we are stepping out of mathematics and into computation. Until now every object in this course has been exact and every operation free: $\mathbb{R}^n$ is infinite, a real number carries infinitely many digits, and writing $\|\mathbf{u}\|$ costs nothing because writing is not computing. A machine has none of that. It has finitely many digits per number, finitely many operations per second, and finite memory, and every one of those limits will eventually bind.

The computations here are small, and modern hardware and libraries are heavily optimised for exactly them. But the habit is what matters: taking a formula apart, asking what each stage costs, and noticing when the same quantity is being computed twice. Do that on a projection and you save a square root. Do it on an algorithm that runs a projection a million times inside a loop, and the difference is the difference between a program that finishes and one that does not.

Looking ahead. Two threads start here. First, we have been counting *exactly*, producing answers like $5n - 1$; usually the constants and the -1 do not matter and only the growth rate does, which is what *asymptotic* notation captures — the reason $5n - 1$ and $8n$ are “the same” in a sense worth making precise. Second, we have assumed a machine stores real numbers. It does not; it stores *floating-point* numbers, of which there are finitely many, and arithmetic on them is not quite the arithmetic of $\mathbb{R}$. Both are next.

1.8. Exercises

In each case, count the operations required, and keep the count *as low as possible* — part of the exercise is noticing what the given information lets you skip.

E1. Let $\mathbf{u}, \mathbf{v} \in \mathbb{R}^{200}$ be orthonormal. Count the operations needed to compute $\text{proj}_{\mathbf{u}}(\mathbf{v})$.

E2. Let $\mathbf{u}, \mathbf{v} \in \mathbb{R}^{50}$ with $\cos(\mathbf{u}, \mathbf{v}) = 1$. Count the operations needed to compute $\|\mathbf{u} - \mathbf{v}\|$.

E3. Let $\mathbf{u}, \mathbf{v} \in \mathbb{R}^{25}$ be nonzero and linearly dependent. Count the operations needed to compute $\cos(\mathbf{u}, \mathbf{v})$.

Answer key

E1. *Zero operations.* “Orthonormal” means both vectors are unit length *and* orthogonal, so $\langle \mathbf{u}, \mathbf{v} \rangle = 0$ is given. Then

$$\text{proj}_{\mathbf{u}}(\mathbf{v}) = \frac{\langle \mathbf{u}, \mathbf{v} \rangle}{\|\mathbf{u}\|^2} \mathbf{u} = \frac{0}{1} \mathbf{u} = \mathbf{0},$$

and the answer is the zero vector, written down without arithmetic. Ignoring the hypothesis and evaluating the optimised formula would have cost $5n - 1 = 999$ operations to arrive at the same $\mathbf{0}$. The lesson is that recognising structure beats computing.

E2. $3n = 150$ operations, and the hypothesis does not help. Directly: $\mathbf{u} - \mathbf{v}$ costs $n = 50$ subtractions, and taking the norm of the result costs $2n = 100$, for $3n = 150$ in total.

It is worth seeing why the given information is no use. Since $\cos(\mathbf{u}, \mathbf{v}) = 1$ the two vectors point the same way, and expanding as in the vector unit’s §13 gives $\|\mathbf{u} - \mathbf{v}\|^2 = \|\mathbf{u}\|^2 - 2\langle \mathbf{u}, \mathbf{v} \rangle + \|\mathbf{v}\|^2 = (\|\mathbf{u}\| - \|\mathbf{v}\|)^2$, so $\|\mathbf{u} - \mathbf{v}\| = \|\mathbf{u}\| - \|\mathbf{v}\|$. That is a genuine shortcut in appearance, but it costs two norms and a subtraction: $2n + 2n + 1 = 4n + 1 = 201$, which is *more* than 150. A shortcut is only a shortcut if it is cheaper.

E3. *1 operation.* Linearly dependent and nonzero means parallel (vector unit, §18.4), so $\mathbf{v} = \alpha \mathbf{u}$ for some $\alpha \neq 0$, and the angle between them is 0 or π . Hence $\cos(\mathbf{u}, \mathbf{v}) = \pm 1$, with the sign being the sign of α . To find that sign, pick any index i with $u_i \neq 0$ and form $\alpha = v_i/u_i$: one division. Then $\cos(\mathbf{u}, \mathbf{v}) = +1$ if $\alpha > 0$ and -1 if $\alpha < 0$.

Evaluating the definition instead would cost one dot product, two norms, one mult and one division: $(2n - 1) + 2n + 2n + 1 + 1 = 6n + 1 = 151$.

2. How costs scale: asymptotic analysis

Section 1 produced exact counts: n , $2n - 1$, $2n$, $5n - 1$. Exactness is satisfying but it is rarely what we want to know. In computer science the question is almost always how the count *scales* — what happens to the cost as the input grows.

2.1. What an algorithm is

Definition 2.1 (Algorithm, informally). An **algorithm** is a finite sequence of steps aimed at solving a problem. It usually has an **input** and always produces an **output**.

Vector addition is about as simple an algorithm as exists. Its input is the pair $\mathbf{u}, \mathbf{v} \in \mathbb{R}^n$; its steps are the n componentwise adds; its output is the vector $\mathbf{w} = \mathbf{u} + \mathbf{v}$.

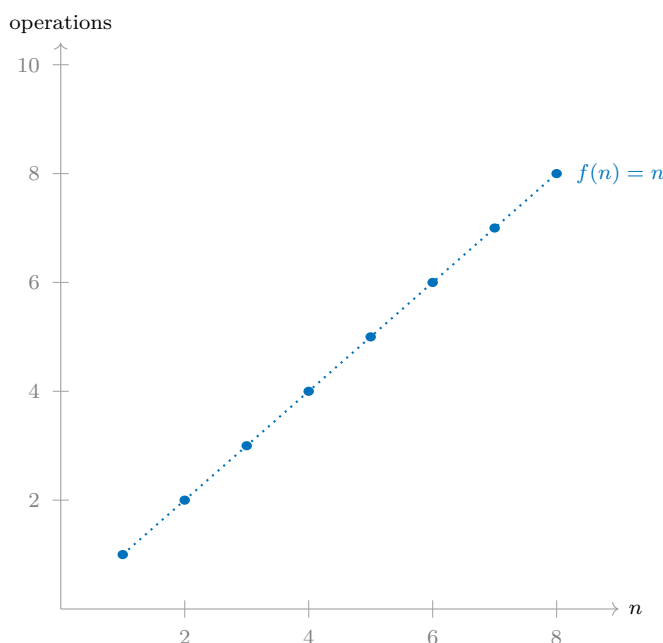
Remark. Every algorithm emits an output — an algorithm that produced nothing would have solved nothing. An input, though, is optional: an algorithm that prints the first thousand primes takes no input at all, and simply runs.

2.2. Operation count as a function of input size

To speak about scaling we need to say what is growing. For an algorithm on vectors, the **input size** is the number of components, n . So write the operation count as a function of it:

$$f(n) = n,$$

where n is the number of components and $f(n)$ is the exact number of operations vector addition performs. Note that n ranges over the natural numbers — an input size is a count, and there is no vector in $\mathbb{R}^{2.7}$ — so the graph of f is a set of isolated points, not a curve.

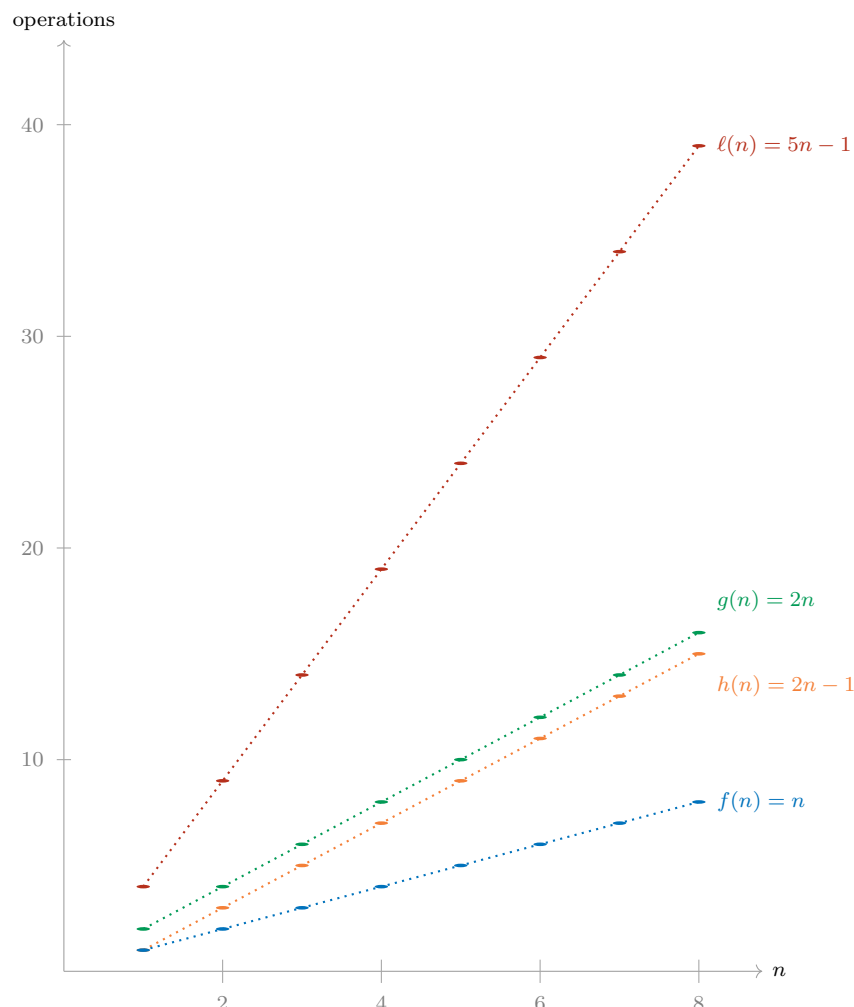


The dotted line is interpolation, drawn only so the eye can see the pattern; the algorithm exists only at the dots. That pattern — the relative geometry of f — is what we mean by the **scale** of the algorithm, and here it is plainly *linear*. The number of operations vector addition performs scales linearly with the number of components.

2.3. The other algorithms

We already have counts for three more. Writing each as a function of input size:

$$\underbrace{g(n) = 2n}_{\|\mathbf{u}\|}, \quad \underbrace{h(n) = 2n - 1}_{\langle \mathbf{u}, \mathbf{v} \rangle}, \quad \underbrace{\ell(n) = 5n - 1}_{\text{proj}_{\mathbf{u}}(\mathbf{v}) \text{ computed efficiently}}.$$



What do the four have in common? Every one of them is a straight line. The slopes differ and one is shifted down by 1, but the *shape* is identical: all four algorithms scale linearly with the number of components. Make the input as large as you like and that does not change — a line stays a line.

2.4. Big-O notation

Since the shape is what matters and the constants are not, we want notation that records the shape and discards the rest.

For a count that scales linearly in the long run we write $O(n)$, read “big-oh of n ”, and call it the algorithm’s **runtime complexity** — also its **asymptotic runtime** or **time complexity**.

So vector addition is $O(n)$. And so are the norm, the dot product, and the projection, despite their different constants and shifts. Big-O is concerned only with the underlying scale, which is set by the **dominant term** of the count.

A common confusion (We are not measuring time). “Time complexity” is a misleading name, and it is worth being clear about it early. We are counting *operations*, not seconds. How long a program actually takes depends on the processor, the language, the compiler, how the data sits in memory,

and what else the machine is doing — two $O(n)$ implementations can easily differ by a factor of a hundred in wall-clock time. What Big-O describes is how the operation count *grows*, and that is a property of the algorithm rather than of any machine that runs it.

2.5. Finding the dominant term

For a polynomial count the dominant term is the highest-degree one. Suppose an algorithm's count is modelled by

$$f(n) = 3n^2 + 2n + 3.$$

As n grows, $3n^2$ outpaces $2n$ and 3 so completely that they stop mattering, and the coefficient 3 does not change the shape — a parabola stretched is still a parabola. So f scales quadratically and we write $f(n)$ is $O(n^2)$.

When the terms are not both polynomials it can be less obvious which one wins, and a table of values helps.

Example 2.1 (A table that nearly misleads). Consider $f(n) = 2^n + n^5$.

n	2^n	n^5	larger
1	2	1	2^n
2	4	32	n^5
5	32	3,125	n^5
10	1,024	100,000	n^5
15	32,768	759,375	n^5
20	1,048,576	3,200,000	n^5
22	4,194,304	5,153,632	n^5
23	8,388,608	6,436,343	2^n
25	33,554,432	9,765,625	2^n
30	1,073,741,824	24,300,000	2^n

Read only the first six rows and you would conclude that n^5 dominates. It does not. The two cross between $n = 22$ and $n = 23$, and past that 2^n pulls away without limit — by $n = 30$ it is more than forty times larger, and the gap keeps widening. So the dominant term is 2^n , and

$$f(n) = 2^n + n^5 \quad \text{is} \quad O(2^n).$$

A common confusion. That example is the reason a table of values is a hint and not a proof. An exponential eventually beats every polynomial, no matter how large the polynomial's degree — but “eventually” can be far to the right of where you stopped looking. Big-O is a statement about the long run, so a handful of small values can point the wrong way.

Remark (A tip: compare with a ratio). A quick way to settle which of two positive functions dominates is to look at their ratio as n grows. If

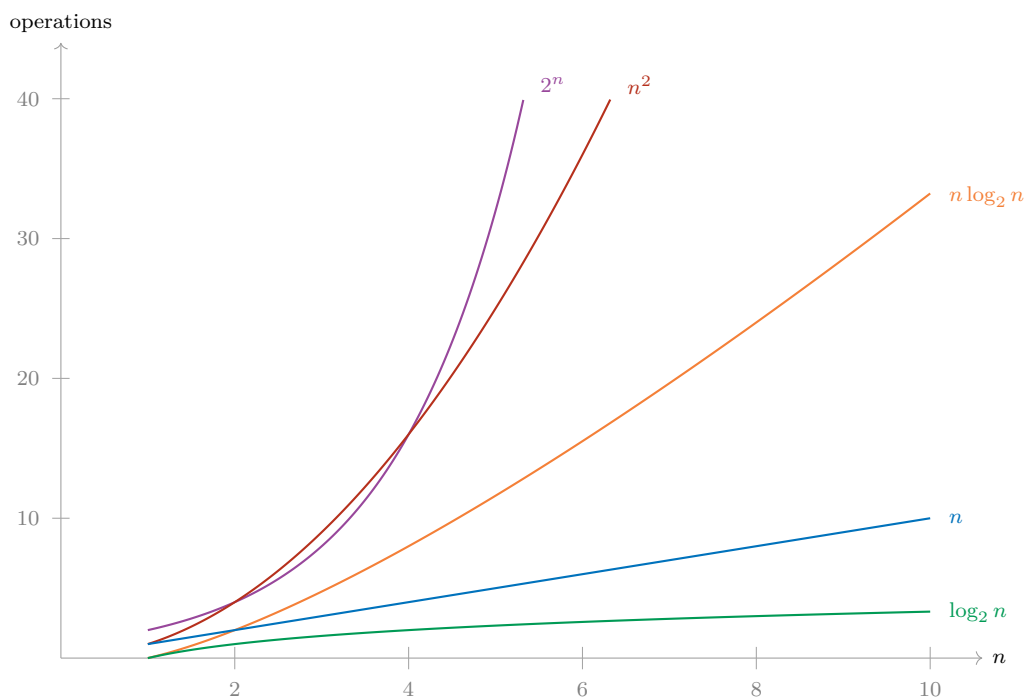
$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0,$$

then f is negligible against g , so g is the dominant one; if the limit is ∞ , then f dominates; and if it is a positive constant, the two scale alike. Two standard facts from Calculus I settle most cases immediately: exponentials outgrow all polynomials, and all polynomials outgrow logarithms. For the example above, $n^5/2^n \rightarrow 0$, confirming 2^n as dominant without any table.

2.6. Complexities you will meet

A few growth rates account for most algorithms in practice.

Complexity	A typical algorithm
$O(1)$	reading one entry of a vector
$O(\log_2 n)$	binary search in a sorted index
$O(n)$	vector addition, the dot product, the norm
$O(n \log_2 n)$	a good general-purpose sort
$O(n^2)$	a naive sort; a matrix–vector product
$O(n^3)$	naive matrix multiplication; Gaussian elimination
$O(2^n)$	checking every subset of n items



The vertical spread at the right-hand edge is the whole point. At $n = 10$ the logarithm has reached about 3 while 2^n passed 40 before n even reached 6. Choosing a better algorithm is not a small optimisation like the ones in §1.6; it changes which problems are solvable at all.

2.7. The formal definition

Everything so far has been informal — “dominant term”, “in the long run”. Here is the precise statement.

Definition 2.2 (Big-O). For functions f and g , we say $f(n)$ is $O(g(n))$ if and only if there exist constants $C > 0$ and $n_0 \geq 0$ such that

$$|f(n)| \leq C |g(n)| \quad \text{for all } n \geq n_0.$$

Every function we will use counts operations, so all of them are positive, and the absolute values may be dropped:

$$f(n) \leq C g(n) \quad \text{for all } n \geq n_0.$$

Read the definition as licensing two concessions. The constant C says we do not care about a fixed multiplicative factor; the threshold n_0 says we do not care what happens for small inputs. Proving f is $O(g)$ therefore amounts to *exhibiting* a C large enough and an n_0 far enough right that the inequality holds from there on. You do not need the smallest such C — any one that works will do.

2.8. Three proofs

Example 2.2. Show that $f(n) = 2n^2 + 2n$ is $O(n^2)$.

For every $n \geq 1$ we have $n \leq n^2$, hence $2n \leq 2n^2$, and so

$$f(n) = 2n^2 + 2n \leq 2n^2 + 2n^2 = 4n^2.$$

Take $C = 4$ and $n_0 = 1$. ■

Example 2.3. Show that $f(n) = n^4 + n^3$ is $O(n^4)$.

For every $n \geq 1$ we have $n^3 \leq n^4$, so

$$f(n) = n^4 + n^3 \leq n^4 + n^4 = 2n^4.$$

Take $C = 2$ and $n_0 = 1$. ■

Both follow the same recipe: bound every lower-order term by the dominant one, then collect. That recipe handles any polynomial.

Example 2.4 (Challenge). Show that $1^2 + 2^2 + 3^2 + \cdots + n^2$ is $O(n^3)$.

Here there is no highest-degree term to bound against, because the number of terms itself grows with n . The trick is to bound each term by the largest one. Every index k in the sum satisfies $k \leq n$, hence $k^2 \leq n^2$; and there are exactly n terms. Therefore

$$\sum_{k=1}^n k^2 \leq \underbrace{n^2 + n^2 + \cdots + n^2}_{n \text{ terms}} = n \cdot n^2 = n^3.$$

Take $C = 1$ and $n_0 = 1$. ■

2.9. A note on what Big-O does and does not say

Complexity notation comes in several forms. Big-O is an *upper* bound on growth — it says the cost is no worse than a certain shape. There are companions for the other directions: Ω for a lower bound, and Θ when upper and lower bounds match. For this course Big-O is enough.

Remark. Big-O is often described loosely as “the worst case”, and for our purposes nothing goes wrong in saying so. Strictly, though, two separate ideas are being run together. *Worst, best* and *average case* concern which *input* you measure; Big-O concerns how a count *grows* once you have chosen. One can perfectly well speak of an average-case $O(n \log_2 n)$.

For the algorithms in §1 the distinction happens not to arise: vector addition performs exactly n adds whatever the entries happen to be, and the same holds for the dot product, the norm and the projection. Their cost depends only on n , so best, average and worst case coincide. Algorithms where they differ — searching, sorting, elimination with pivoting — are where the vocabulary earns its keep.

Big-O is ubiquitous well beyond this course: it is the common language of software engineering, numerical analysis and scientific computing, and it is how the cost of every algorithm in the rest of this unit will be reported.

Looking ahead. One assumption remains unexamined. Throughout §1 and §2 we have counted operations on *real numbers*, as though a machine could hold one. It cannot: there are infinitely many reals and only finitely many patterns of bits. What a computer actually stores is a *floating-point* number, and arithmetic on those is not quite the arithmetic of $\mathbb{R}$ — sums can depend on the order you add them, and quantities that should cancel exactly do not. That is the next topic, and the last of this lecture.

3. The set a computer actually works in

3.1. Every subject starts with a set

Mathematical subjects share a structural pattern, and it is worth naming before we disturb it. In each one there is a *set* of objects and a collection of *operations* or rules that act on them. Everything else — the questions, the observations, the conjectures, the theorems — grows out of asking what happens to those objects under those rules in one scenario after another.

Calculus is a clear case. The underlying set is the real numbers, together with functions built from them. The operations on functions are governed by properties of $\mathbb{R}$, and out of those properties comes the limit; out of the limit comes the derivative; and out of the derivative comes the rest of the subject. Nothing in that chain was available until the set was fixed.

Underlying set	Some operations	What grows from it
$\mathbb{R}$, and functions on it	limits, derivatives, integrals	calculus, real analysis
$\mathbb{C}$	complex arithmetic, conjugation	complex analysis
$\mathbb{Z}$	divisibility, congruence, counting	number theory, combinatorics
$\mathbb{R}^n$, later matrices	addition, scaling, the dot product	linear algebra

The underlying set plants the seed for a whole garden of mathematical knowledge.

Remark. The vector unit was exactly this pattern, carried out in front of you. We fixed the set — $\mathbb{R}^2$, later $\mathbb{R}^n$ — then fixed the operations, and then spent five parts asking what followed: when do two vectors span, when are they independent, what does orthogonality buy, which subsets are closed. None of those questions could even be *stated* before the set and the operations were on the table.

3.2. A computer works in a different set

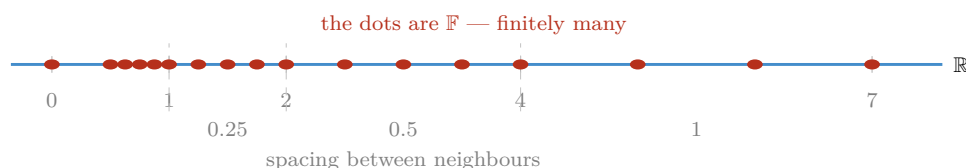
Here is the disturbance. When a computer tackles a mathematical problem — and especially a *continuous* one, a problem posed over the real numbers — it does not work in $\mathbb{R}$. It works in a peculiar set of its own.

Definition 3.1 (Floating-point numbers, informally). The **floating-point numbers**, written $\mathbb{F}$, are a *finite* set of numbers used to approximate the real numbers, each carrying only a fixed, finite number of digits.

Two consequences follow immediately, and both matter.

- $\mathbb{F}$ is *finite*, while $\mathbb{R}$ is not merely infinite but uncountable. So $\mathbb{F} \subset \mathbb{R}$: every floating-point number is a real number, and almost no real number is a floating-point number. The machine is working inside a very sparse subset of the set the problem was posed in.
- The elements of $\mathbb{F}$ are *not evenly spaced*. They crowd together near zero and spread apart as the magnitude grows, because a fixed number of digits buys finer resolution for small numbers than for large ones.

The picture below shows both facts at once. It uses a miniature floating-point system — three digits of precision in base 2, with only four exponents — so that the whole set fits on the page; a real system has vastly more points but is spaced in exactly this pattern.



Read the spacing across the picture: it *doubles* at every power of two. Between 1 and 2 the representable numbers are 0.25 apart; between 2 and 4 they are 0.5 apart; between 4 and 8 they are 1 apart. Near zero they are packed tightly, and far from zero they are strewn thinly. The line is continuous; the dots are all the machine has.

Everything in $\mathbb{R}$ that is not a dot must be *rounded* to the nearest dot before the machine can hold it. That rounding is small, and for a single number it is harmless. The question — which is what makes this a topic rather than a footnote — is what happens when millions of such roundings accumulate through an algorithm like the ones we counted in §1.

3.3. Floats and doubles

You have almost certainly met this set already. Programming languages expose it directly, and name the members by their **precision** — the number of digits each one carries:

Name	Storage	Roughly
float, single precision, float32	32 bits	about 7 decimal digits
double, double precision, float64	64 bits	about 16 decimal digits

A `double` is the default nearly everywhere it matters: it is what Python’s `float` is, what MATLAB uses unless told otherwise, and what numerical libraries assume. A `float` halves the storage and the precision, which is a real trade in graphics and machine learning and a liability almost everywhere else.

The word *precision* is the one to hold on to. It does not mean the numbers are approximately right in some vague sense; it means each one carries a specific, countable number of digits, and everything past that is gone.

3.4. Base 2

One more ingredient before we can say what a floating-point number actually *is*. A classical computer stores everything — numbers included — as patterns of 0s and 1s, which is to say in **binary**.

Binary is not exotic. It is simply a numerical *base*, base 2, and we have been conditioned by a decade of schooling to think in base 10 instead — for no better reason than that people have ten fingers. In any base b , a string of digits is shorthand for a sum of multiples of powers of b :

	Place values	Value
237_{10}	$2(10^2) + 3(10^1) + 7(10^0)$	$200 + 30 + 7 = 237$
1011_2	$1(2^3) + 0(2^2) + 1(2^1) + 1(2^0)$	$8 + 0 + 2 + 1 = 11$

The same idea continues to the right of the point, with negative powers:

$$0.101_2 = 1(2^{-1}) + 0(2^{-2}) + 1(2^{-3}) = \frac{1}{2} + \frac{1}{8} = 0.625.$$

So the toy system in the figure above was nothing more than this: a few binary digits, scaled by a power of two.

Looking ahead. A base is a choice of bookkeeping, and normally an innocent one — 0.625 is 0.625 whether you write it in base 10 or as 0.101_2 . But the choice is not always innocent, because a fraction that terminates in one base need not terminate in another. That is where the surprises live, and it is where we pick up next.